CH 2 — Classes

Luke Lu • 2026-10-20

Intro to Class and Objects

> 📘 **Definition — Classes and Objects**
>
> Imperative Programming
>
> - Procedural Programming: data + standalone functions
> - Object-Oriented Programming: put the data and the functions that operate that data within unit or construct
>   - The construct is called **class**
>   - An instance/value of a class is called an **object**

Example:

A simple class called Student(Program "06-classes/version1.cc/student.cc"):

```
struct Student{
  int assignment, mt, final;
  float grade();
}

float Student::grade(){ //In the scope of student Class, the function grade
   return 0.4 * assignment + 0.2 * mt + 0.4 * final;
}
```

Note: in C++, `struct` can have functions within, and C++ `structs` are classes

Program "06-classes/version2.cc" is the more formal and preferred way of usage.

```
Student billy{90, 55, 45}; //C-style  initialization
//billy is a student object

float res1 = billy.grade(); //grade() has access to billy's marks

Student lily{95, 80, 85};

cout << lily.grade(); //grade() has access to lily's mark
```

> 📘 **Definition — Methods**
>
> A "function" part of a class is called "**methods**", which is commonly used
>
> A **method** can only be called by using an **object**
>
> Within the **method**, the method has access to the fields of that object.
>
> All **methods** have a hidden parameter named **`this`**, which is a pointer to the object use to call the method

Example:

```
billy.grade(); //"this" is equiv to "&billy", or interpret that their address is same
```

For the function, `grade` is actually

```
 float grade(){
  return this->assignment * 0.5 + this->mt * 0.2 + this->final 0.4;
};
```

We assume `this->` if not present and identifier undeclared

Object Initialization

In C++, we can write special methods to initialize objects

Previously,

1. C-style Initialization if there is no explicit constructor
   - Memory allocation
   - Field initialization in declaration order with parameter values in declaration order
     - In case of under-initialization, system does default initialization

       non-class/primitive data $\Longrightarrow$ assigned to `0`
2. C++ style Initialization if there is at least one explicit constructor
   - Memory allocation
   - Field initialization in declaration order
     - Primitives are not initialized
     - Classes are being called with **default constructors**
   - **Constructor** calls
     - Cannot default initialize in C-style
     - Logging
     - Constructor/function overloading
     - Sanity checks, restriction on input

> 📘 **Definition — Constructors Rules**
>
> - Same name as the class name
> - No return type
> - Default constructor does not have brackets upon calling
>
>   ```
>   CON a constructor(); //ILLEGAL
>   CON b constructor; //LEGAL
>   ```

```
struct Student{
  //...
  Student(int assign, int mt, int final){
    this->assign = assign;
    this->mt = mt;
    this->final = final;
  }
}
```

Member-Initialization List:

- field insitialization in declaration order of C++ style for fields named in the list
- use member-initilization list for all initializations
- use constructor for sanity checks, logging, etc.
- MIL processed in field declaration order. Not in MIL order.

```
struct id{
  string name;

  id;
  id(string name){ this->name = name;}
}
struct posn{
  int x, y;
  id uid;
  posn(int x, int y, string name): y{y*2}, x{x+1}, uid{name+ "?"} {}
}
```

For **ONE** parameter constructors:

```
struct Node {
  int data;
  Node *next;
  Node(int data):data{data}, next{nullptr}{}
}
Node m{4};
Node m = 4; //implicit conversion
foo(m);
foo(4); //One parameter constructor create implicit conversions
```

Note: implicit conversion for one parameter constructors could be useful and also be troublesome. We can disable implicit conversion by keyword: `explicit`

```
struct Node {
  int data;
  Node *next;
  explicit Node(int data):data{data}, next{nullptr}{}
}
Node m{4};
Node m = 4; //ILLEGAL
foo(m);
foo(4); //ILLEGAL
```

Class Methods

A class comes with a default constructor and the **Big 5**:

- Copy Constructor
- Copy Assignment Operator
- Destructor
- Move Constructor
- Move Assignment Operator

Definition — Copy Constructor

Copy Constructor creates a copy that is a new instance. It is used

- when constructing an object as a copy of another object
- called when an object is passed by value
- called when an object is return by value

Example:

1. Default implementation

```
//default hidden implementation
Student::Student(const Student &other) mt{other.mt}, final{other.final} {}
//must pass by reference
//Sometime, the default implemenation is not suitable

Student s1{90, 98, 33};
Student s2{s1}; //s2 = s1; the copy constructor
```

2. Custom implementation

```
struct Node {
  int data;
  Node *next;
  Node (int data, Node *next): data{data}, next{next}{}
  Node(const Node &other): data{other.data}, next{other.next}{}

  Node::Node(const Node &other) : \
  data{other.data}, next{other.next?new Node {*other.next}:nullptr;} {}

};

Node *p new Node{1, new Node {2, new Node {3, nullptr}}}
//p->1->2->3->null

Node m{*p}; //copy constructor the default

Node *q = new Node{*p}; //copy constructor
// Before having custom implementation
//m->1->2(of p)->3(of p)->null(of p)
//q->1->2(of p)->3(of p)->null(of p)

// After having custom implementation
//m->1->2->3->null
//q->1->2->3->null
```

📘 **Definition — Destructor**

Every class has **exactly one** destructor

- cannot overload destructor
- takes no parameters
- For a class `T`, its destructor is written as

  `~T(){}`
- no return type
- default destructor body is empty
- sometimes the default destructor is not correct

When an object is destroyed, the destructor is called

When are objects destroyed:

- When a stack allocated object goes out of scope
- When call `delete` on the address of a healp allocated object

Steps for object destruction

1. Destructor body runs
2. Fields that are objects are destroyed (by calling their destructor)
3. Spaces are reclaimed

Example:

```
Node *p  = new Node{1, new Node{2, new Node{3, nullptr}}}; //p->1->2->3->Null

//If we do nothing, we leak 3 node memory
//Note p is not leaked b/c it is in stack as it stores the address to the first node.
//If we write `delete p;`, we are destroying node 1 only, leaking node 2, 3


//Destructor

Node::~Node(){delete next;}

delete p;
//calls delete node 1 -> delete node 2 -> delete node 3 -> delete nullptr
//then space for node 3 reclaimed -> reclaimed node 2 -> reclaimed node 1

Node m{1, new Node{2, nullptr}};
// `m` will be deleted automatically and leak no memory because it is on stack
Node n{3, &m};
// `n` will cause error b/c m is on stack, but C++ expects a node in heap memory
```

Definition — Copy Assignment Operator

Assignment operator

- updates and existing object rather than creating a new instance.
- is implemented as a method
- the default implementation does field for field assignment

Example:

1. Copy constructor vs Assignment operator

```
Student s1{50, 12, 32};
Student s2{s1}; //copy constructor
Student s3 = s1; //copy constructor
Student s4{50, 23, 45};
s3 = s2; //copy assignment operator
```

2. Implementation for `Node`

```
n1 = n2; //n1.operator= (n2);
n1 = n2 = n3; //assignment cascades, so operator= must return a Node reference

Node &Node::operator=(const Node &other){
  if(this == &other) return *this; //self-assignment check, avoid dangling pointer
  data = other.data
  delete next;
  next = other.next?new Node{*other.next}:nullptr;
  //if `new` fails to allocate memory, `next` will not be updated
  //this will cause `next` to be a dangling pointer
  //it would be better to have "all or nothing" behaviour which will be discussed
in future
  return *this; //or return others;

//corrected
Node &Node::operator=(const Node &other){
  if(this == &other) return *this;
  Node *temp{next};
  next = other.next?new Node{*other.next}:nullptr;
  data = other.data;
  delete temp;
  return *this; //or return others;
}

//another way, Copy and Swap Idiom
import<utility>; //for swap()
void Node::swap(Node &other){
  using std::swap;
  swap(data, other.data);
  swap(next, other.next);
}
Node &Node::operator=(const Node &other){
  Node temp{other};
  swap(temp); //swap method from Node
  //DO NOT write delete temp;, temp is in stack
}
```

Definition — Move Constructor

Refer to “lectures/10-rvalue/node.cc”

Calling `oddsOrEvens` results in 6 basic constructor calls

Returning `evens` will result in 3 copy constructors due to copy elision.

Copy Elision: C++ allows(sometimes requires) that extra copies be avoided

An **r-value** reference is a reference to a temporary value on an object that is about to be destroyed.

If we know that the RHS of a copy is a r-value, then instead of copying we could hijack/steal in hopes of being more efficient.

```
Node::Node(Node &&other): data {other.data}, next {other.next} {
  other.next = nullptr;
}
```

After this implementation, it will only call 1 copy constructor.

Definition — Move Assignment Operator

Refer to “lectures/10-rvalue/node.cc”

```
Node m = ...;
m = oddsOrEvens();

//say other is 1->3->5, and m is ?->?->?->? and the goal is to make m be 1->3->5

Node &Node::operator=(Node &&other){
  using std::swap;
  swap(data, other.data);
  swap(next, other.next);
  return *this;
}
```

If a move constructor or a move assignment operator has been implemented, the compiler will choose it whenever the RHS is a r-value or a l-value that is about to be destroyed.

If not, it will use the copy constructor and copy assigntment operator.

Copy/Move Elision

C++ allows/requires copy/move operations even if this changes observable behaviours.

```
Vec makeVec(){return {4, 5};}

Vec v = makeVec();// one basic constructor call, no copy/move constructor


void foo(Vec v){...}

foo(makeVec()); //one basic constructor, no copy/move constructor
```

Important to know that copy or move elision can happen, not required to memorize situations if when it is required or optional

> 💡 **Tip — Rule of Five**
>
> If implement a custome version of any of the Big Five, custom versions of all five are typically needed.

Operator overloading

C++ requires some operators to be overloaded as methods:

- `operator=`
- `operator[]`
- `operator->`
- `operator()`
- `operator T()`

It is best to implement I/O operator as standalone functions

```
struct Vec{
  int x;
  int y;

  Vec operator+(const Vec &rhs){
  // `this` represents the LHS
  return {x+rhs.x, y+rhs.y};
  }

  Vec operator*(const int k){
    return {k * x, k * y};
  }
}
//Standalone function because `int` is not class `Vec`
Vec operator*(const int k, const Vec &rhs){
  return {rhs * k};
}

Vec v1;
Vec v2;
Vec v3 = v1+v2;
Vec v4 = v1 * 3;
Vec v5 = 3 * v1;//must be implemented as a standalone function
```

Arrays of Object

```
struct Vec{
  int x, y;
  Vec(int x, int y);

};

//No default constructor since we wrote 2 parameter constructor
Vec arr1[10];
Vec *arr2 = new Vec[10];
//This will not compule since when an array of objects is constructed, each object is
default constructed(by default constructor)
```

Option1: Provide a default constructor.

Option2: Stack array: initialize each object explicitly: `Vec arr1[] = {Vec(1, 2), ...};`

Option3(Most Common): Create array of pointers to objects:

```
Vec *arr1[10] //Stack array

vec **p = new Vec *[10]; //Heap array
```